Python Code

```
"""
The Kevin Bacon Game.

Replace "pass" with your code.
"""

import simpleplot
import comp140_module4 as movies


class Queue:
    """
    A simple implementation of a FIFO queue.
    """
    # intialize the object (queue)
    def __init__(self):
        """
        Initialize the queue.
        """
        self._queue = []
    # return length of the queue
    def __len__(self):
        """
        Returns: an integer representing the number of items in the queue.
        """
        return len(self._queue)

    # return string representatioj
    def __str__(self):
        """
        Returns: a string representation of the queue.
        """
        return str(self._queue)
    # add element to queue
    def push(self, item):
        """
        Add item to the queue.

        input:
            - item: any data type that's valid in a list
        """
        self._queue.append(item)
    # remove element from queue
    def pop(self):
        """
        Remove the least recently added item.

        Assumes that there is at least one element in the queue.  It
        is an error if there is not.  You do not need to check for
        this condition.
```

```
51
52          Returns: the least recently added item.
53          """
54          removed = self._queue.pop(0)
55          return removed
56      # remove all elements from queue
57      def clear(self):
58          """
59          Remove all items from the queue.
60          """
61          self._queue.clear()
62
63
64
65  def bfs(graph, start_node):
66      """
67      Performs a breadth-first search on graph starting at the
68      start_node.
69
70      inputs:
71          - graph: a graph object
72          - start_node: a node in graph representing the start node
73
74      Returns: a two-element tuple containing a dictionary
75      associating each visited node with the order in which it
76      was visited and a dictionary associating each visited node
77      with its parent node.
78      """
79      # intiailize
80      dist = {}
81      parent = {}
82      # more intializing
83      for node in graph.nodes():
84          dist[node]  = float('inf')
85          parent[node] = None
86
87      # set up starting node
88      dist[start_node] = 0
89      queue = Queue()
90      queue.push(start_node)
91
92      # go through whole queue
93      while len(queue) > 0:
94          current_node = queue.pop()
95          # iterate over neighbors of the current node and set their distance
96          for nbr in graph.get_neighbors(current_node):
97              if dist[nbr] == float('inf'):
98                  dist[nbr] = dist[current_node] + 1
99                  parent[nbr] = current_node
100                 # add to queue
101                 queue.push(nbr)
102
103     return dist, parent
104
```

```
def distance_histogram(graph, node):
    """
    Computes the distance between the given node and all other
    nodes in that graph and creates a histogram of those distances.

    inputs:
        - graph: a graph object
        - node: a node in graph

    returns: a dictionary mapping each distance with the number of
    nodes that are that distance from node.
    """
    # intialize
    histogram = {}
    # get from bfs
    distances, _ = bfs(graph, node)
    # iterate over keys to sum up count for each
    for current_node in distances:
        histogram[distances[current_node]] = histogram.get(distances[current_node], 0)
+ 1
    return histogram

def find_path(graph, start_person, end_person, parents):
    """
    Computes the path from start_person to end_person in the graph.

    inputs:
        - graph: a graph oject with edges representing the connections between people
        - start_person: a node in graph representing the starting node
        - end_person: a node in graph representing the ending node
        - parents: a dictionary representing the parents in the graph

    returns a list of tuples of the path in the form:
        [(actor1, {movie1a, ...}), (actor2, {movie2a, ...}), ...]
    """
    # intialize path with final entry
    path = [(end_person, set())]
    current_node = end_person
    # iterate until we go through whole path
    while current_node != start_person:
        parent_node = parents.get(current_node)
        if parent_node is None:
            # no path possible since the root of graph has been reached before
start_person
            return []
        # add set to list
        path.append((parent_node, graph.get_attrs(parent_node, current_node)))
        # update current node
        current_node = parent_node
    # correct order
    path.reverse()
    return path

def play_kevin_bacon_game(graph, start_person, end_people):
```

```
    """
    Play the "Kevin Bacon Game" on the actors in the given
    graph.

    inputs:
        - graph: a a graph oject with edges representing the connections between
people
        - start_person: a node in graph representing the node from which the search
will start
        - end_people: a list of nodes in graph to which the search will be performed

    Prints the results out.
    """
    # get output from previous methods
    _, parent = bfs(graph, start_person)
     # iterate over actors in end_people
    for end_person in end_people:
        path = find_path(graph, start_person, end_person, parent)
        movies.print_path(path)


def run():
    """
    Load a graph and play the Kevin Bacon Game.
    """
    graph5000 = movies.load_graph('subgraph5000')

    if len(graph5000.nodes()) > 0:
        # You can/should use smaller graphs and other actors while
        # developing and testing your code.
        play_kevin_bacon_game(graph5000, 'Kevin Bacon',
            ['Amy Adams', 'Andrew Garfield', 'Anne Hathaway', 'Barack Obama', \
             'Benedict Cumberbatch', 'Chris Pine', 'Daniel Radcliffe', \
             'Jennifer Aniston', 'Joseph Gordon-Levitt', 'Morgan Freeman', \
             'Sandra Bullock', 'Tina Fey'])

        # Plot distance histograms
        for person in ['Kevin Bacon', 'Stephanie Fratus']:
            hist = distance_histogram(graph5000, person)
            simpleplot.plot_bars(person, 400, 300, 'Distance', \
                'Frequency', [hist], ["distance frequency"])

# Uncomment the call to run below when you have completed your code.

run()

```